

StegoStamp Image Watermarking

Eugene Francisco

June 2026

1 Overview

Watermarking refers to the process of embedding information in generated data so as to label that data as having been generated by a model. This paper introduces StegoStamp, a novel way to watermark images, robust to common image transformations and lossy compression and virtually undetectable to the human eye. Inspired by Bui et al.¹, we train a *message encoder* to embed hidden bit sequences in the latent representation of images from a fixed autoencoder; alongside this, we train a *secret decoder* to recover messages from possibly corrupted images. We use techniques from Tancik et al.² to make these watermarkers robust to spatial transformations like cropping and rotating, as well as regular image transformations like noising, blurring, and JPEG.

2 Related Work

Discuss Bui et al.³ and Tancik et al.⁴ and HiDDeN.

3 Methods

Watermarking derives from the much older process of steganography, where the goal is to conceal messages within another medium. Stated concretely, the goal of image watermarking is to take a so-called *cover image* c and message m and generate a *stego image* $\tilde{c}$ in which m is somehow embedded; later on, a verifier should be able to take $\tilde{c}$ (possibly compressed or transformed in some way) and recover a message $\hat{m}$ which should be similar to m .

3.1 Architecture

StegoStamp trains two neural networks:

- (i) A message encoder F_θ which is used to encode messages and generate watermarked images.

- ¹ **TODO** add reference
- ² **TODO** add reference
- ³ **TODO** reference
- ⁴ **TODO** reference

- (ii) A secret decoder D_φ which is used to recover a message $\hat{m}$ from a (possibly corrupted) $\tilde{c}$.

These networks encode and decode in the following way:

- (i) Message Encoder: given a cover image c of size $H \times W$ and a message m of length ℓ , we first partition c into equal sized patches of size $P \times P$, where we assume H, W are divisible by P . Let $\mathcal{P}$ be those patches. Fix an image autoencoder (E, G) , where E is the encoder that sends images to the latent space and G is a generator that creates images from latent variables.

For each patch $p \in \mathcal{P}$:

- (a) Create latent variable $z_p = E(g)$.
- (b) Create a latent offset $\delta = F_\theta(m)$.
- (c) Create a watermarked patch $\tilde{p} = G(z_p + \delta)$

Stitch together the patches $\tilde{p}$ to get a watermarked image $\tilde{c}$ and return $\tilde{c}$.

The neural network F_θ is a very small convolutional neural network implemented in a similar way to Bui et al.

- (ii) Secret Decoder: Given a (possibly corrupted) watermarked image $\tilde{c}$, the secret decoder returns $\hat{m} = D_\varphi(c)$. The neural network $D_\varphi(c)$ is implemented as a modified ResNet50 with the final flattening layer replaced by a 1×1 CNN whose output is ℓ channeled. An average is taken across the width and height dimensions to retrieve an ℓ length vector of logits which are thresholded and returned as $\hat{m}$.

This last phase *voting* step is taken from Zhu et al.'s HiDDeN.

Here, θ and φ are trainable parameters; note in particular that the autoencoder (E, G) remain fixed and are not trained.

3.2 Training Details

TODO training objectives. We use curriculum learning to train F_θ and D_φ . Learning is split into the following phases:

1. Fix a single minibatch of images from the dataset. Train F_θ and D_φ on this minibatch until bit accuracy of m against $\hat{m}$ crosses 0.9. Sample m 's bits uniformly and randomly. Before passing $\tilde{c}$ into D_φ , we choose with probability $1/2$ whether to crop $\tilde{c}$, in which case we crop $\tilde{c}$ to a random patch of fixed size (in our experiments, $3P \times 3P$).
2. Expose F_θ and D_φ to a much larger portion of the training set (50,000 images in our case). Train until bit accuracy reaches 0.8 as this will likely drop the bit accuracy by a significant amount.

Implementation Details

For simplicity, we trained on square images of size 384×384 using a patch size of $P = 96$. We use a very small neural network for F_θ with only 9,168 parameters.